Smart Residential Parcel Box with RFID Access Control and Cloud Monitoring

Name of Student(s):

Zhuofan Sun

Jiayin Chen

Yangfei Wang

W Booth School of Engineering Practice and Technology

Hamilton, Ontario

Canada

March 2026

Table of Contents

1 INTRODUCTION 3

2 SYSTEM OVERVIEW 4

2.1 SYSTEM OVERVIEW 4

2.2 SYSTEM ARCHITECTURE 4

2.3 WORKFLOW 7

3 HARDWARE DESIGN 7

3.1 COMPONENTS 7

3.2 CIRCUIT DESIGN 8

4 SOFTWARE DESIGN 9

4.1 ARCHITECTURE 9

4.2 VISION AND TRACKING 10

4.3 ACCESS AND LOCKER 11

4.4 EVENTS AND ALERTS 13

4.5 DATABASE STORAGE 13

5 WEB INTERFACE AND SYSTEM INTEGRATION 14

5.1 WEB-BASED INTERFACE 14

5.2 STREAM AND SOCKET 14

5.3 COMMAND AND DATA FLOW 15

6 RESULTS AND DEMONSTRATION 15

6.1 FUNCTIONAL RESULTS 15

6.2 DEMO SCENARIOS 16

Scenario 1: Live vision and tracking 16

Scenario 2: Authorized RFID access 16

Scenario 3: Unauthorized RFID alarm 17

Scenario 4: Hardware button request 17

6.3 BEHAVIOR AND TRADE-OFFS 17

7 SAFETY AND SECURITY 17

8 LIMITATIONS 18

9 CONCLUSION 19

1 Introduction

As e-commerce grows, residential package delivery has become increasingly common [1]. In many cases, packages are left unattended at the front door when homeowners are not present, which creates a significant risk of package theft, commonly referred to as “porch piracy” [2], as well as issues related to unauthorized access and lack of delivery verification.

Traditional parcel boxes provide basic physical storage but lack intelligent features such as identity verification, real-time monitoring, and event logging. As a result, homeowners have limited visibility and control over parcel delivery and pickup activities [3], [11].

To address these challenges, this project proposes an IoT-based smart parcel box system for residential houses. The system integrates multiple sensing, control, and communication components, with a Raspberry Pi serving as the edge computing device, providing a secure and trackable package handling solution. A camera module is used for face detection and tracking people, enabling the system to identify activity near the parcel box. An RFID authentication mechanism is implemented to ensure that only authorized users can access the box. Besides, a servo motor simulates the locking mechanism, while RGB LEDs and a buzzer provide visual and audible feedback. An ultrasonic sensor is used to detect whether the parcel box is occupied by detecting the distance between the sensor and the parcel, enabling the system to update the status of the parcel box.

The system also features a web-based user interface with WebSocket Communication [5], allowing users to remotely monitor system status and send control commands in real time. Furthermore, a local SQLite database is used to store event logs, including access attempts, authorized RFID cards, door sessions and captured images, enabling traceability and historical analysis.

Through the integration of sensing, edge computing, real-time communication, and data storage, this project demonstrates a complete IoT system that enhances the security, usability, and reliability of residential package delivery.

2 System Overview

2.1 System Overview

The proposed system is a smart parcel box designed for residential households, aimed at improving the security and traceability of package delivery and pickup. The system is installed near the entrance of a house and integrates sensing devices, control mechanisms and real-time communication technologies to monitor activity and regulate access.

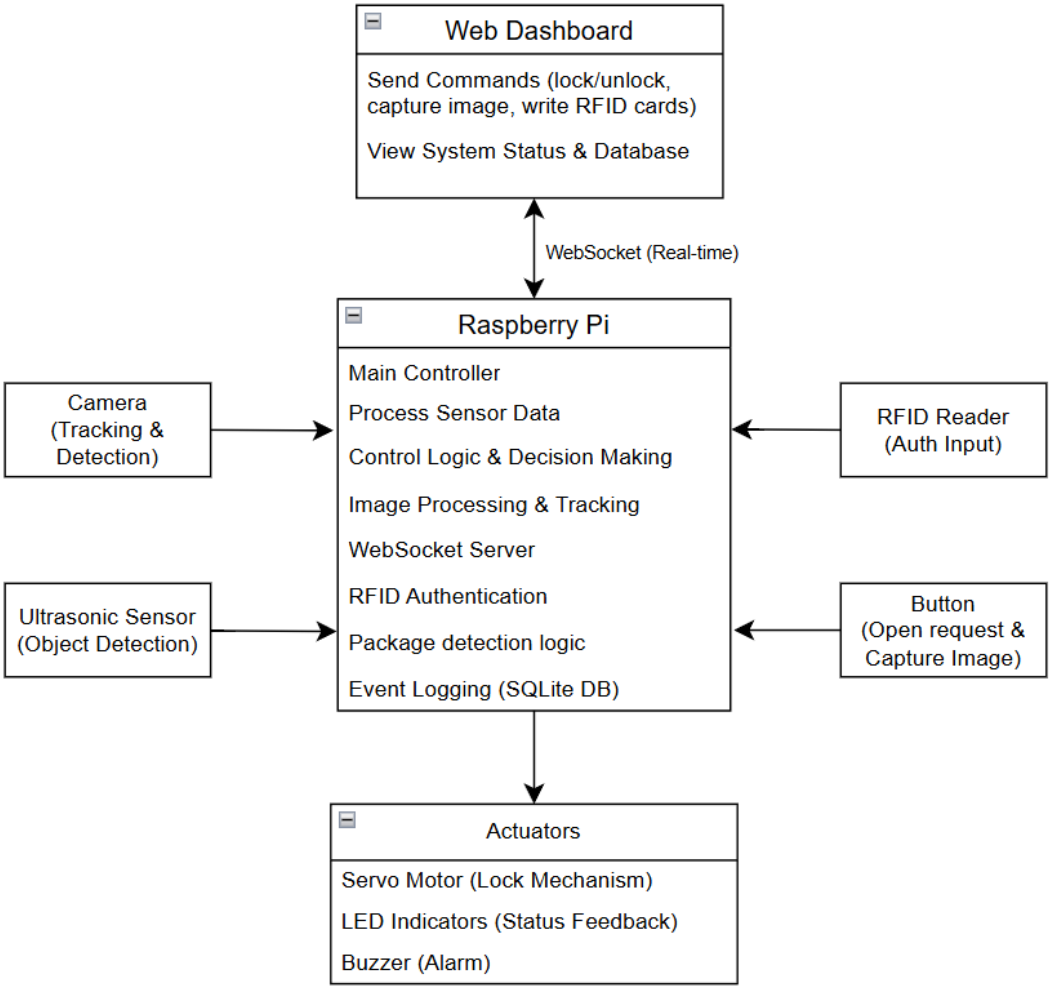
A camera module equipped with a pan-tilt mechanism is used to detect faceand track individuals approaching the parcel box. The system activates when face is detected and prepares for user interaction. To pick up a package, users can authenticate using an RFID tag. In addition, a physical button is provided to allow delivery personnel to request access manually, which also triggers image capture for verification.

An ultrasonic sensor is used to detect the presence of a package inside the parcel box by measuring distance changes, which enables automatic identification of delivery events without requiring manual input.

The system is controlled by a Raspberry Pi, which acts as the edge computing device, processing sensor data, executing control logic, and coordinating system responses. A web-based dashboard allows users to monitor system status, live video of the camera, and send commands in real time through WebSocket communication. Besides, a local SQLite database is used to store event logs, including access attempts, captured images, verified RFID tags and door events, enabling persistent data storage and traceability.

2.2 System Architecture

The system is designed using a layered IoT architecture [4], which separates sensing, processing, communication and user interaction into distinct components. This modular design improves system clarity, scalability and maintainability.



2.2.1 Input Devices (Sensing and User Input Layer)

This layer includes all components responsible for detecting environmental conditions and receiving user input:

- Camera Module: Performs face detection and target tracking near the parcel box.
- RFID Reader: Provides authentication input by reading user RFID tags.
- Ultrasonic Sensor: Detects whether a package is present inside the parcel box.
- Button: Allows delivery personnel to send a local access request and triggers image capture.

These components provide input signals to the central controller but do not directly control system outputs.

2.2.2 Edge Layer (Raspberry Pi)

The Raspberry Pi serves as the central processing unit of the system and is

responsible for:

- Processing sensor data from input devices
- Performing image processing and tracking
- Handling RFID authentication
- Executing control logic and decision making
- Managing WebSocket communication
- Recording system events using a local SQLite database

All system decisions are made at this layer, ensuring that centralized and modular control architecture.

2.2.3 Communication Layer (WebSocket)

The system uses WebSocket-based communication to enable real-time, bidirectional communication between the Raspberry Pi and the web dashboard [5]. Unlike traditional HTTP request-response models, WebSocket maintains a persistent connection allowing instant transmission of user commands, real-time system status updates and efficient event-driven communication [5].

2.2.4 Application Layer (Web Dashboard)

The application layer consists of a web-based dashboard that allows users to interact with the system. The dashboard provides the following functionalities:

- Sending control commands (e.g., lock/unlock, capture image, etc.)
- Viewing system status and activity logs
- Monitoring captured images

This layer serves as the primary interface between the user and the system.

2.2.5 Output Layer (Actuators)

The output layer includes components responsible for executing actions based on system decisions:

- Servo Motor: Controls the locking and unlocking mechanism of the parcel box.
- LED Indicators: Provides visual feedback for system status (e.g., face detected, access granted/denied)
- Buzzer: Generates audible alerts for unauthorized access attempts.

All actuators are controlled exclusively by the Raspberry Pi and do not interact with input devices.

2.3 Workflow

The system operates through an event-driven workflow, consisting of detection, authentication, decision-making, and response.

When a person approaches the parcel box, the camera detects face and begins tracking the target using the pan-tilt mechanism. The system enters an active state, and visual indicators be activated to blue meaning it is searching for people and face.

The ultrasonic sensor monitors the internal state of the parcel box after door is closed. When a change in distance is detected, the system determines whether a package has been placed or removed. This event is recorded in the system log.

Users can initiate access in two ways; By RFID authentication, the user scans an RFID tag, which is verified against authorized credentials. Or, by button request, the user presses a physical button to request access. When pressed, the system captures an image and logs the request for verification, an email notification will also be sent to owner's phone.

Based on the authentication results or system logic, when access granted, the servo motor unlocks the parcel box, a green LED is activated, and an image is captured to record the event. Otherwise, the system activates a red LED and triggers the buzzer alarm. An image is also captured to record the unauthorized attempt.

All events, including face detection, package detection, access attempts, and captured images, are stored in a local SQLite database. At the same time, relevant information is transmitted to the web dashboard through WebSocket communication, allowing real-time monitoring and control.

3 Hardware Design

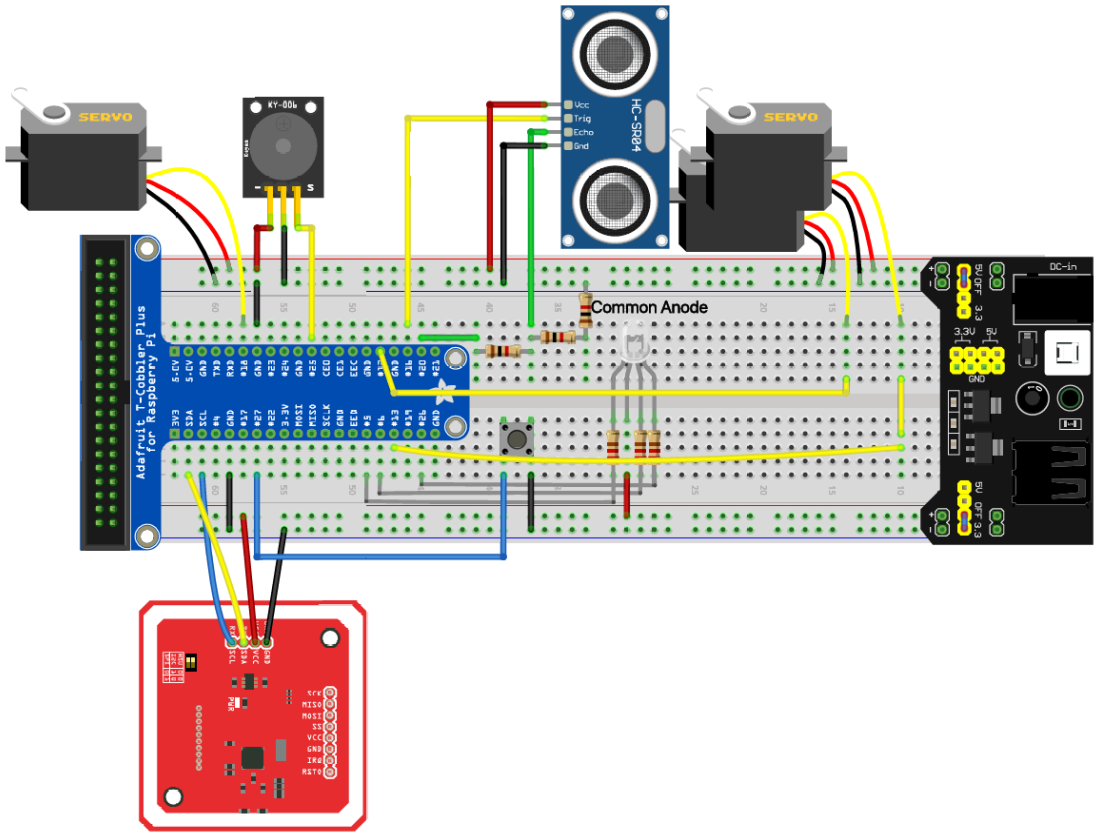
3.1 Components

The system consists of the following key hardware components:

- Raspberry Pi: Serves as the central controller and edge computing device. It processes sensor inputs, executes control logic, and manages communication with the web interface.
- Camera Module (OV5647 Pan-Tilt Mounted): Used for face detection, target tracking, and image capture. The pan-tilt mechanism allows the camera to follow approaching users.
- RFID Reader (PN532): Provides user authentication by reading RFID tags. Only authorized users are granted access to the parcel box [6].
- Ultrasonic Sensor (HC-SR04): Measures the distance inside the parcel

box to detect the presence or absence of a package [8].

- Push Button: Acts as a local input device that allows users to request access. When pressed, the system captures an image and logs the event.
- Servo Motors (SG90): Used to simulate the locking mechanism of the parcel box.
- LED Indicators (RGB LED): Provide visual feedback for system status, such as face detection, access granted, and access denied.
- Buzzer Module: Generates an audible alarm for unauthorized access attempts.
- Breadboard Power Supply Module: Provides stable 3.3V and 5V power rails for the system components.



3.2 Circuit Design

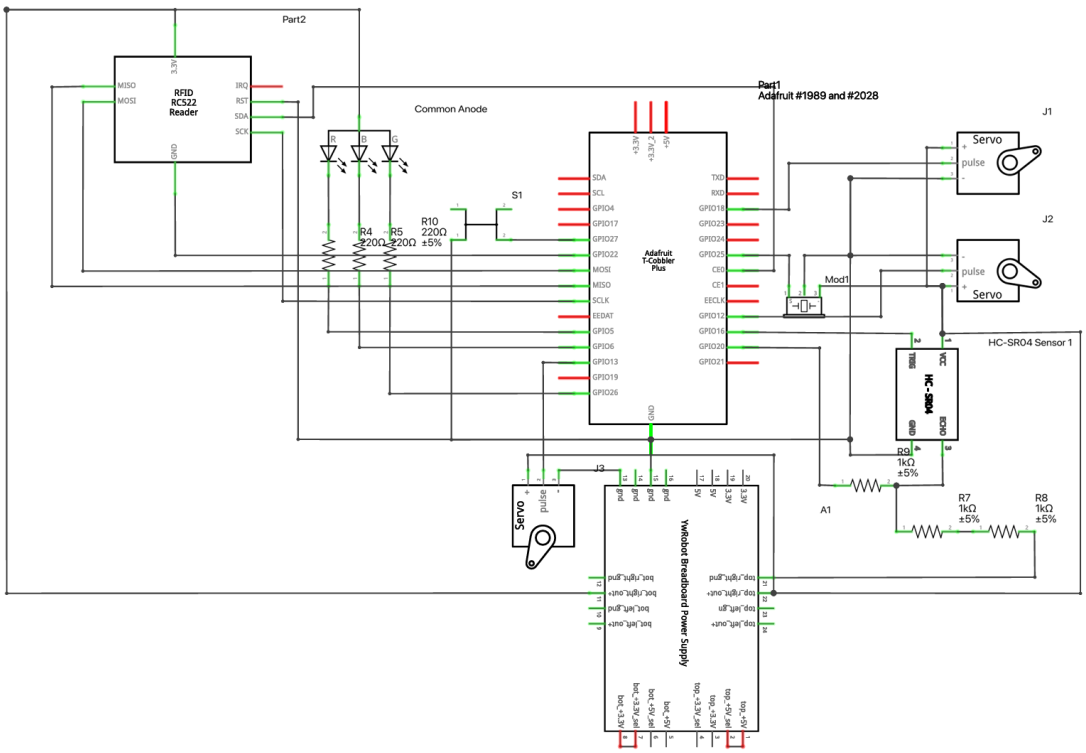
The electrical connections between components are illustrated below. The Raspberry Pi is connected to peripheral devices through its GPIO pins, using standard communication protocols and digital signals.

The RFID reader communicates with the Raspberry Pi using the I2C interface [6], [7]. The ultrasonic sensor uses separate triggers and echo pins to measure distance. A voltage divider is implemented on the echo line to safely interface with the Raspberry Pi's 3.3V GPIO pins [7], [8].

Servo motors are controlled using PWM signals generated by the Raspberry Pi. Due to their relatively high current requirements, they are powered through the external breadboard power supply to ensure stable operation.

The RGB LED is configured as a common-anode circuit with current-limiting resistors to prevent damage. Each color channel is controlled through separate GPIO pins. The buzzer is connected to a digital output pin and is activated when an alarm condition is triggered.

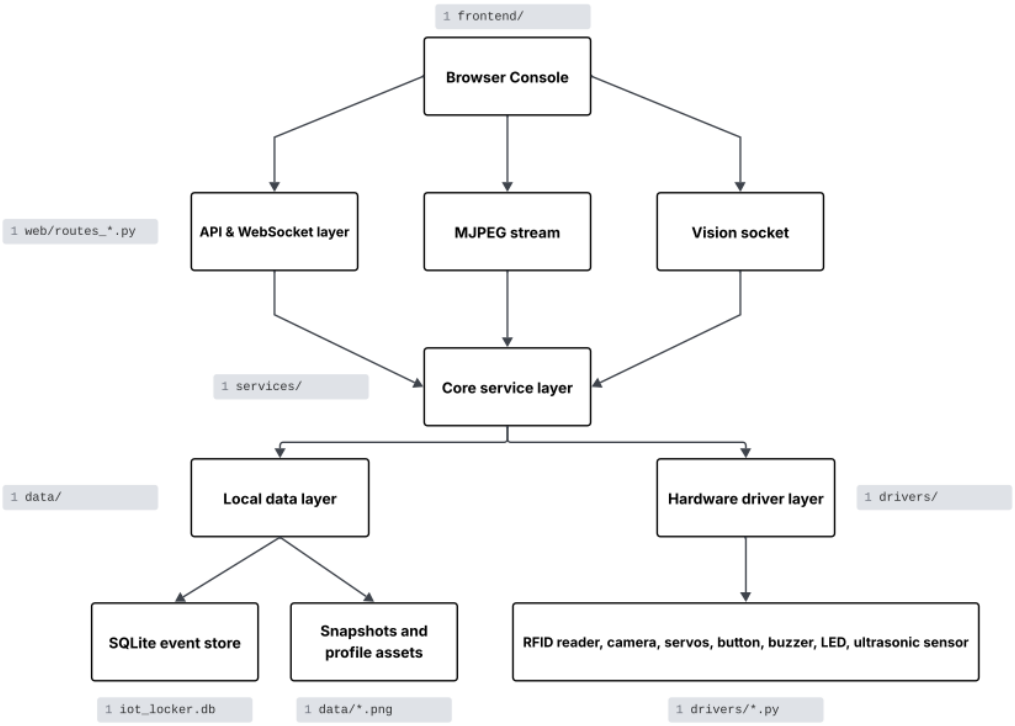
The push button is connected as a digital input with appropriate pull-up or pull-down configuration, allowing the Raspberry Pi to detect user interaction reliably.



4 Software Design

This chapter explains how ParcelBox is organized as a complete device-side system. The focus is not on individual files. It is on how the browser, backend, services, storage, and hardware work together on one Raspberry Pi.

4.1 Architecture



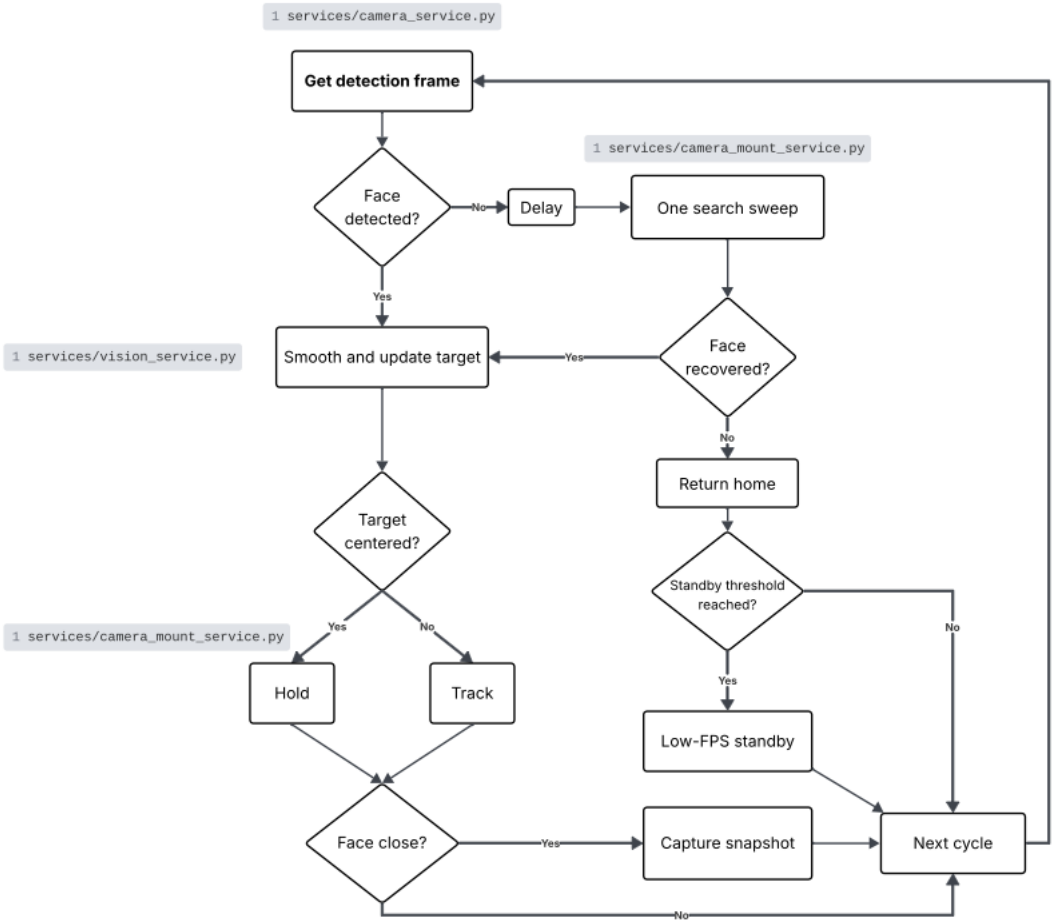
Overall software architecture of the ParcelBox system.

ParcelBox runs as a single local system on a Raspberry Pi. The browser console is used for control and status checking. The backend provides HTTP APIs, live video access, and real-time state updates. Behind it, the service layer handles access control, vision, locker actions, alerts, snapshots, and event processing. Local storage keeps records and settings on the device.

The software is divided into clear layers. The browser layer handles display, commands, and settings. The web layer exposes interfaces to the browser, but the main logic stays in the service layer. Below that, the data layer manages persistence and the driver layer handles direct hardware access. This keeps responsibilities clear and avoids mixing UI logic, route logic, and hardware control in one place.

This structure is suitable for a Raspberry Pi prototype. It reduces coupling, makes debugging easier, and supports later extension. It also matches the project scope: one device, local state, and no external cloud platform or remote database.

4.2 Vision and Tracking



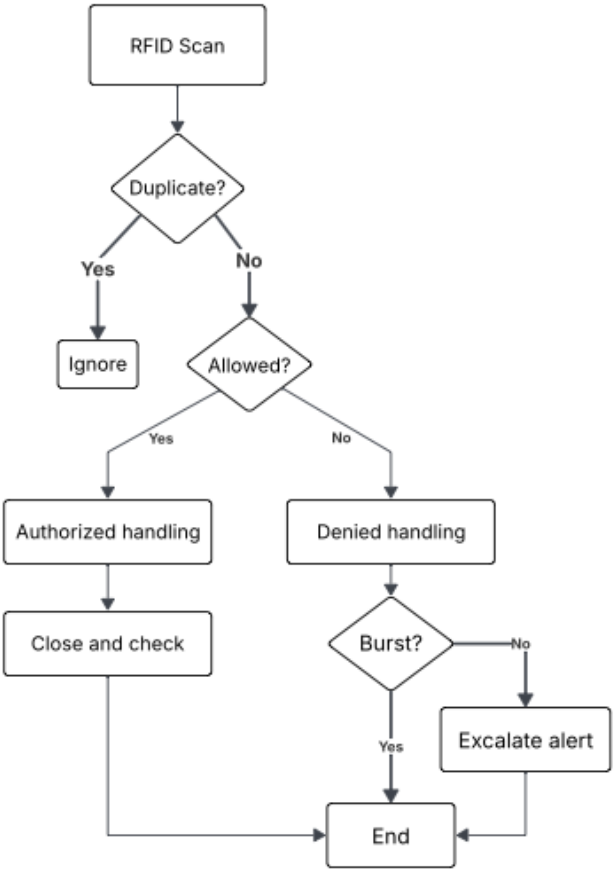
Vision and tracking flow are used by the system.

The vision part of ParcelBox is designed for continuous operation. The camera supports both browser preview and detection. One path is used for live video and snapshots, while another is used for face detection and target updates. This split helps keep the interface responsive while still supporting tracking.

When a face is detected, the system smooths the box and updates the target before moving the camera mount. This improves stability and avoids sudden motion. If the target is still centered, the mount can hold its position. If the face moves, the mount follows it.

When the face disappears, the system does not start a large search immediately. It first keeps a short hold, then enters a lower activity state, and later performs a limited search if needed. If the target is still not found, the mount returns home. This behavior reflects a practical trade-off: the project focuses on simple face tracking and stable runtime behavior rather than heavier recognition features.

4.3 Access and Locker



RFID access and locker control workflow.

The RFID process is designed as a full control loop. It relies on a PN532 driver extended from the Adafruit PN532 library, which is compatible with most common 13.56 MHz high-frequency cards and emulated cards used in practice (ISO14443A). After a card is scanned, the system reads the UID and filters duplicate scans. It then checks whether the card is allowed to open the locker based on the stored access rules.

If access is allowed, the system can capture a snapshot, open the locker, and create the related records. The workflow continues after opening: the door is closed automatically, then occupancy is checked with the ultrasonic sensor. This means the system records not only the access event, but also the state of the locker after the access session ends.

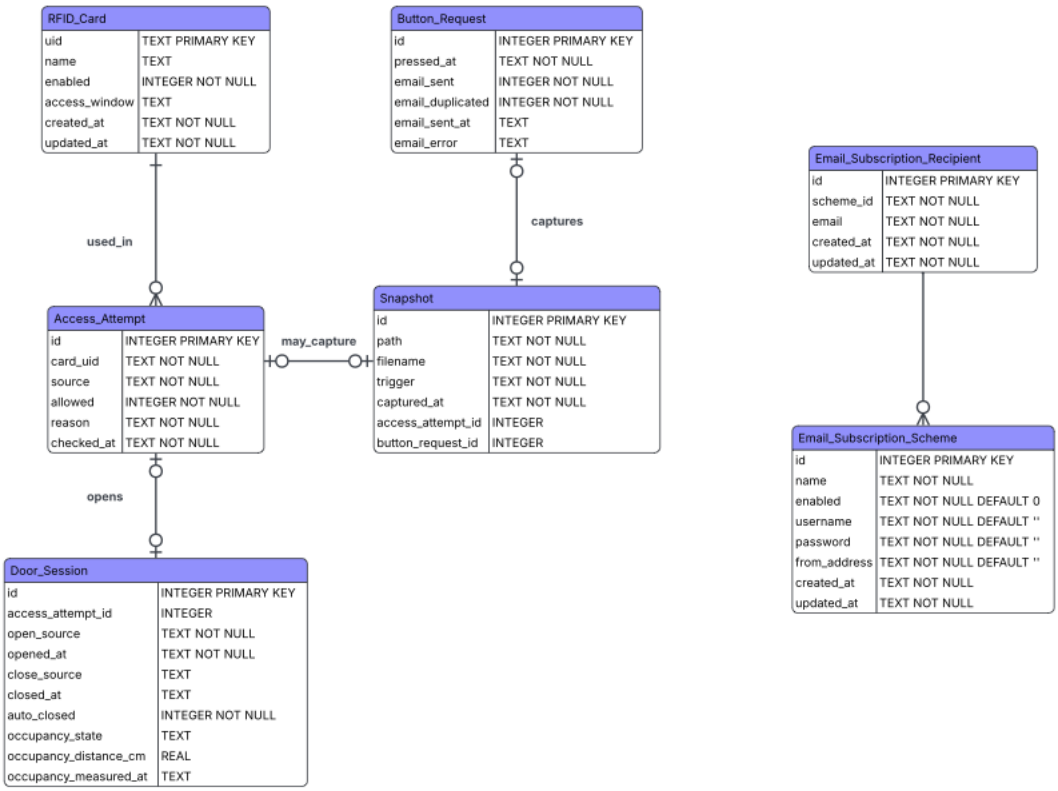
If access is denied, the event is still recorded, and a local warning is raised. Repeated denied scans within a short time window can trigger a stronger alert. This makes the RFID module more than a simple open command. It becomes a small access management loop with checking, logging, warning, and final state update.

4.4 Events and Alerts

ParcelBox keeps a structured event stream rather than simple console logs. Access attempts, button actions, snapshots, and alerts are stored in a form that can be reviewed in the browser. This helps both operation and debugging, because the user can compare recent events with live device status and camera output.

Alerts are also controlled rather than fully reactive. Warning sounds, stronger alarms, and front-end notifications depend on repeated conditions and cooldown rules. This reduces noise and keeps the device from reacting too aggressively to single accidental inputs.

4.5 Database Storage



Data model for events, access records, and settings.

The system uses local SQLite storage because it is a single-device prototype [9]. This choice keeps deployment simple, supports offline use, and makes debugging easier. It fits the project scale better than a remote database.

The stored data includes RFID cards, access attempts, door sessions, button requests, snapshots, device profile data, and email settings. These records support both daily operation and later review. They also connect related actions, such as an access attempt and its door session, or a button request and its snapshot.

Snapshot files are kept separately from database records, which keeps image

handling practical on a small device. User-editable settings are stored persistently, so the web console works as both a monitoring tool and a configuration interface.

5 Web Interface and System Integration

This chapter explains how the browser and the Raspberry Pi form one working system. The web interface is not only for display. It is the main control surface, status view, and verification tool for the prototype.

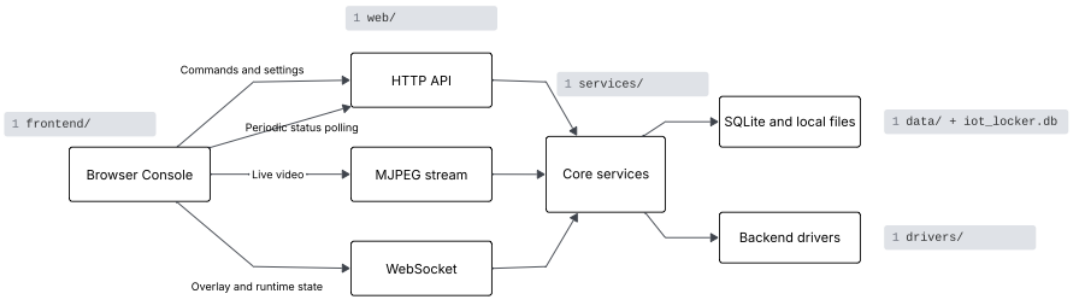
5.1 Web-Based Interface

The front end of ParcelBox is an operation and debugging console. Its role is to show the live view, current device state, recent events, and stored snapshots, while also giving the user control over actions and settings.

The interface is divided into several practical areas. The overview page combines the live stream, state cards, and control actions. Cards and access pages support RFID management. Events and snapshots pages support checking results after use. Debug and data pages expose raw tables and system information, while settings pages allow profile and email updates.

The front end stays lightweight by using plain web technologies served directly by the backend. This reduces build complexity and suits a small embedded system. Even so, it still provides useful features such as live overlay, notifications, snapshot viewing, and persistent settings.

5.2 Stream and Socket



Browser and device communication channels.

The browser communicates with the device through two main real-time channels. MJPEG is used for the live video stream, while WebSocket is used for overlay data and runtime state [5]. This separation is important because video frames and structured state updates behave differently and are easier to manage when they are not mixed together.

With this design, the browser can show live video while also drawing face boxes

and displaying tracking state. It also allows status changes, such as standby or search behavior, to appear quickly without changing the video transport itself.

The design is also practical for stability. The browser can reconnect after interruption, and the backend can manage socket cleanup more clearly. Using MJPEG plus WebSocket is a deliberate trade-off: it is simpler than a heavier media stack and works well for a local Raspberry Pi prototype.

5.3 Command and Data Flow

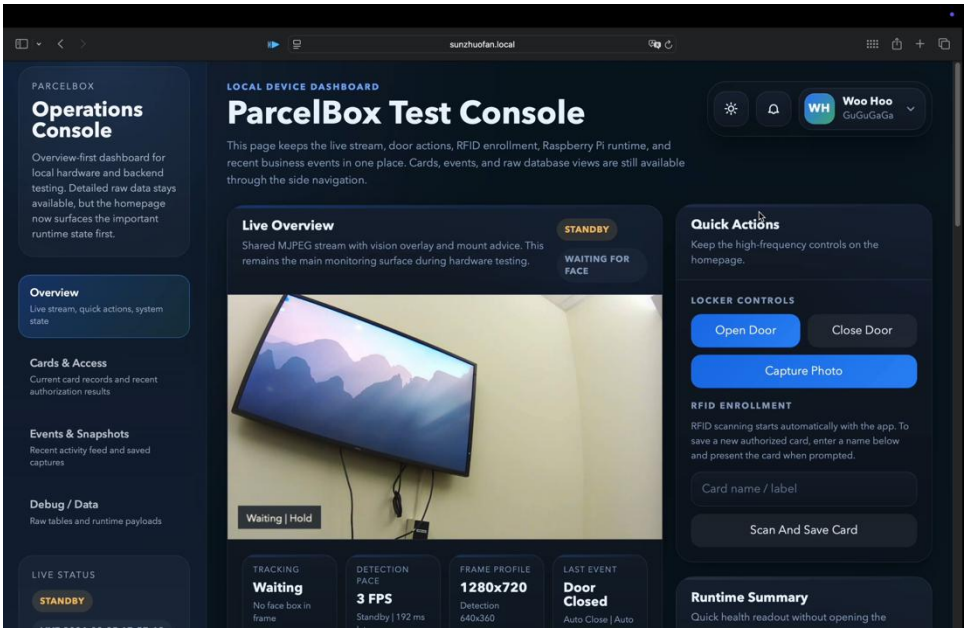
The browser does two jobs at the same time. It receives status through polling, live video, and socket updates, and it also sends commands to the device. These commands include opening or closing the locker, taking snapshots, managing cards, silencing alerts, and updating settings.

This gives ParcelBox both a monitoring plane and a control plane in one interface. A user can trigger an action, watch the hardware response, and then confirm the result in the event history. That tight feedback loop is one of the strongest parts of the whole system integration.

6 Results and Demonstration

This chapter summarizes the completed prototype, shows the main demo cases, and discusses the most important design trade-offs.

6.1 Functional Results

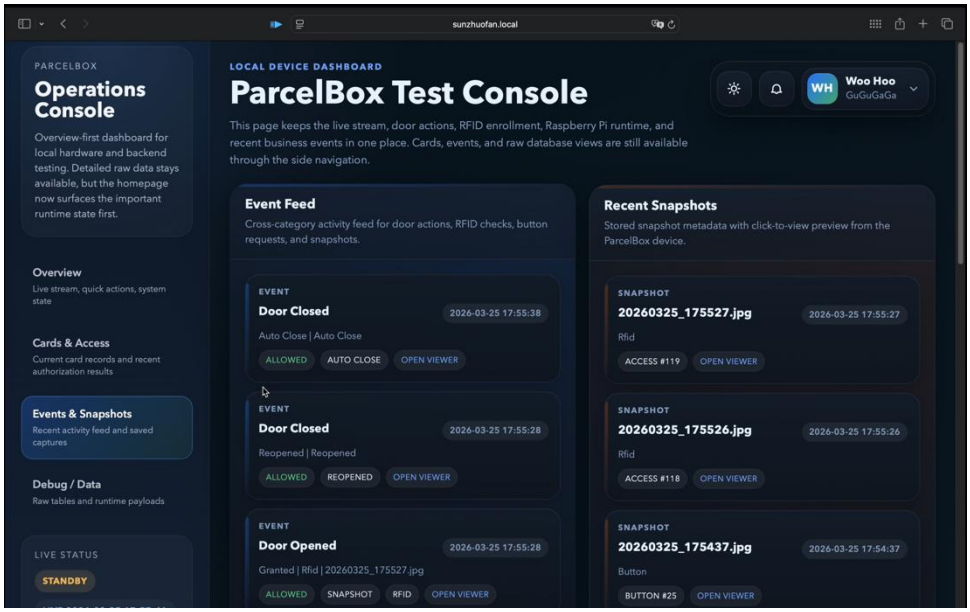


The final ParcelBox prototype combines several functions on one Raspberry Pi device: RFID-based access control, live vision and face tracking, snapshots, button-triggered requests, local alerts, event storage, and a browser console for

control and review. These features are linked through shared services and stored records, so the result is a connected system rather than a group of isolated functions.

In practice, the prototype can detect a user, respond to RFID scans, open and close the locker, check occupancy after access, save local records, and display all of this in the browser. This makes it a clear end-to-end embedded prototype rather than a set of independent tests.

6.2 Demo Scenarios



The system can be demonstrated through four short scenarios that show the main behavior of the final prototype.

Scenario 1: Live vision and tracking

A person stands in front of the device and moves within the camera view. The browser shows the live stream, face overlay, and tracking state while the mount follows the target and later returns to a waiting state. This demonstrates the full link between camera input, detection, mount control, and front-end feedback.

Scenario 2: Authorized RFID access

A valid RFID card is scanned at the reader. In our tests, the PN532-based RFID workflow worked with a standard MIFARE Classic 1K card, a MacCard, and an Apple Wallet NFC-emulated transit card. These tests showed that the PN532-based RFID workflow can work with several common 13.56 MHz cards and emulated cards in practice. However, some higher-security cards may use non-static or random UIDs, which makes them less suitable for a UID-based access model [10]. Once an enrolled card is recognized, the locker opens, related records are created,

the door closes after the timer, and occupancy is checked at the end of the session. This shows the full loop from access decision to final locker state.

Scenario 3: Unauthorized RFID alarm

An invalid or blocked card is scanned, or denied scans are repeated within a short window. The system records the denial, plays a warning, and can escalate the alert state when the condition repeats. This demonstrates that denial and alert behavior are managed as part of the system rather than handled as a simple rejection.

Scenario 4: Hardware button request

The physical button is pressed on the device. The system creates a request event, may capture a snapshot, and updates the event feed in the browser. This shows how a local hardware action becomes a visible event in the wider workflow.

6.3 Behavior and Trade-offs

The final system shows several useful runtime behaviors. It can recover the browser stream after interruption, reduce activity when no face is present, avoid repeated search stacking, and limit repeated snapshot or alert actions through cooldown and burst rules. These choices improve stability and make testing easier.

In our longest continuous test run of 50 minutes, CPU usage stayed around 75% to 85% during active video streaming and face detection and dropped to around 25% to 35% during standby periods. Memory usage stayed below 1 GB, and CPU temperature remained below 50 degrees Celsius. These observations support the value of the standby mechanism and suggest that the current design is practical for a Raspberry Pi prototype.

The project also makes clear trade-offs. It uses face detection and tracking rather than recognition. It uses MJPEG plus WebSocket rather than a heavier media stack. It uses local SQLite rather than remote storage. These decisions reduce complexity and fit the goal of a single-device prototype.

The result is a system that is easier to deploy, debug, and explain. At the same time, the current design is not meant for public deployment, and longer hardware soak testing is still needed. Even so, the final prototype shows effective integration across sensing, control, storage, and browser interaction.

7 Safety and Security

Safety and security are major design priorities in this project because the system is intended to protect parcels from unauthorized access while operating safely in a residential environment. The parcel box uses RFID authentication as the main

access control method for package retrieval. Only registered RFID cards are allowed to unlock the door. If an unregistered card is detected, the system treats the event as an unauthorized access attempt and immediately activates the red LED indicator and buzzer alarm. At the same time, the system can capture and store an image of the event for later review. This helps improve security by combining identity verification, visual evidence, and real-time alerts.

The project also improves parcel protection through internal status monitoring. An ultrasonic sensor is installed inside the parcel box to determine whether a package is present, whether the box is empty, or whether the door is not properly closed. In our system logic, a measured distance within 20 cm indicates that an object is present, a distance between 20 cm and 50 cm indicates that the box is empty, and a distance greater than 50 cm indicates that the door is open or not locked correctly. By combining this sensing information with RFID access records and remote door control events, the system can detect suspicious parcel removal and unsecured door conditions more effectively.

From the safety perspective, the system is designed to reduce the risk of accidental misuse. The door is controlled by a servo lock and automatically closes after a short delay, which helps prevent the parcel box from being left open unintentionally. In addition, the entire prototype is built around a single Raspberry Pi with clearly defined hardware modules, including the camera, RFID reader, ultrasonic sensor, buzzer, LED, and servo motor. This integrated design keeps the system compact and easier to manage during testing. However, safe wiring, stable power supply, and careful mounting of moving components such as the servo and door mechanism are still required to ensure reliable operation.

8 Limitations

Although the project demonstrates the core idea of a smart residential parcel box, it still has several limitations. First, this system is currently a single-device prototype running locally on one Raspberry Pi. It is intended to validate the complete workflow of hardware control, event logging, and local web monitoring, but it is not yet designed as a large-scale commercial product with multiple users, cloud-based account management, or advanced deployment support.

Second, the access control method relies mainly on RFID card authentication and does not yet include stronger identity verification methods such as facial recognition, mobile authentication, or encrypted cloud authorization. The current vision module performs face detection and tracking only, rather than true identity recognition. As a result, the system can observe and record activity, but it cannot

fully confirm the identity of a person from the camera alone.

Another limitation is that the ultrasonic package detection logic still depends on threshold calibration. The current rules are practical for the prototype, but the actual measurement may vary depending on parcel size, sensor placement, box dimensions, and door position. For this reason, the reliability of package detection and door-state judgment may be affected if the hardware layout changes. The README also indicates that real-box ultrasonic threshold calibration and long-duration soak testing are not yet fully completed.

Finally, the system currently emphasizes local functionality rather than hardened network security. The local web console and monitoring features are useful for demonstration and development, but they still require further security improvement before real residential deployment. For example, stronger authentication, secure remote access, improved alert delivery, and more robust fail-safe handling would be needed before the system could be considered production ready.

9 Conclusion

In conclusion, this project presents a practical smart parcel box prototype that combines package monitoring, access control, and theft prevention into one integrated Raspberry Pi system. The design supports two main use cases: delivery personnel can request access so the owner can remotely allow package drop-off, and authorized users can retrieve parcels through RFID verification. At the same time, the camera, web interface, email notification, ultrasonic sensor, LED, and buzzer work together to provide event recording, status monitoring, and alarm response.

The project successfully demonstrates that a low-cost IoT device can be used to improve the security of residential parcel delivery. Compared with a traditional parcel box, this system adds controlled access, monitoring capability, and abnormal-event detection, which makes package storage safer and more intelligent. It also shows how hardware devices and local software services can be integrated into a complete edge-based solution.

Overall, the prototype meets the main project objective of protecting delivered parcels from unauthorized access while giving the homeowner better visibility and control. Although there are still limitations in calibration, scalability, and long-term deployment readiness, the current system provides a strong foundation for future improvement. With additional refinement, the project could be extended into a more robust and practical smart home security product.

Reference

[1] U.S. Census Bureau, “Quarterly Retail E-Commerce Sales Report,” Mar. 10, 2026. [Online]. Available: https://www.census.gov/retail/eCommerce.html.

[2] B. Stickle, M. Hicks, A. Stickle, and Z. Hutchinson, “Porch pirates: examining unattended package theft through crime script analysis,” Criminal Justice Studies, vol. 33, no. 2, pp. 79-95, 2020, doi: 10.1080/1478601X.2019.1709780.

[3] M. Cieśla, “Perceived importance and quality attributes of automated parcel locker services in urban areas,” Smart Cities, vol. 6, no. 5, pp. 2661-2679, 2023, doi: 10.3390/smartcities6050120.

[4] H. Mrabet, S. Belguith, A. Alhomoud, and A. Jemai, “A Survey of IoT Security Based on a Layered Architecture of Sensing and Data Analysis,” Sensors, vol. 20, no. 13, Art. no. 3625, 2020, doi: 10.3390/s20133625.

[5] Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Dec. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6455.

[6] NXP Semiconductors, “PN532 User Manual,” UM0701-02, Rev. 2, Nov. 5, 2007. [Online]. Available: https://www.nxp.com/docs/en/user-guide/141520.pdf.

[7] Raspberry Pi Ltd., “GPIO and the 40-pin header,” Raspberry Pi Documentation. [Online]. Available: https://www.raspberrypi.com/documentation/hardware/raspberrypi/gpio/.

[8] ELECFreaks, “Ultrasonic Ranging Module HC-SR04,” datasheet. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf.

[9] SQLite.org, “Appropriate Uses For SQLite.” [Online]. Available: https://www.sqlite.org/whentouse.html.

[10] NXP Semiconductors, “MIFARE product and handling of UIDs,” AN10927, Rev. 4.1, Jul. 5, 2018. [Online]. Available: https://www.nxp.com/docs/en/application-note/AN10927.pdf.

[11] J. Z. Ooi and C. C. Tan, “Smart Modular Parcel Locker System using Internet of Things (IoT),” in 2021 IEEE 11th Int. Conf. on System Engineering and Technology (ICSET), Shah Alam, Malaysia, 2021, pp. 66-71, doi: 10.1109/ICSET53708.2021.9612542.